

DNS service for hybrid cloud

Problem Statement: With the increasing use of virtualization, everyone tends to host a service in container/VM. The default public DNS is not able to resolve this newly created names located in containers. To make this transition from public to private and across different cloud providers, we are proposing a DNS service which will work securely and smoothly in case of Hybrid Cloud.

Project Description

1. What is DNS

Domain Name System in simple words is the Internet's system which translates Human readable domain names to IP addresses which Machines use to communicate with each other. With this resolution now the web browser in our computer will know where to go to reach the hostname or domain name which we want to go. Without DNS, we would have to type the series of four numbers and dots into your browser to retrieve the website.

2. What is Hybrid Cloud

Hybrid cloud is a cloud computing environment that uses a mix of on-premises, private cloud and third-party, public cloud services with orchestration between the two platforms. By allowing workloads to move between private and public clouds as computing needs and costs change, hybrid cloud gives businesses greater flexibility and more data deployment options.[1]

3. Building network using containers

A) What are containers

In simple words, a container is a miniature version of the virtual machine, with essential dependencies only and all features of a virtual machine such as isolation, operational efficiency along with easy, reliable and quick deployment, portability and version control. A container provides more granular control over resources which eventually improves the efficiency of the infrastructure[2]

B) Managing containers cluster using kubernetes

Now that we have seen what is containers, we need a container-orchestration system for automating deployment, scaling, and management of containerized applications and Kubernetes is an open-source system we will be using. It was originally designed by Google and is now maintained by the Cloud Native Computing Foundation. It aims to provide a

"platform for automating deployment, scaling, and operations of application containers across clusters of hosts".It works with a range of container tools, Example Docker.

Relative Study

1. Amazon route 53

Amazon "Route 53" is a scalable and highly available Domain Name System (DNS). It is part of Amazon.com's cloud computing platform, Amazon Web Services (AWS). Since the servers hosting Route53 are distributed across the globe, it can be used for routing inside the AWS cloud as well as outside the AWS cloud, which is an ideal example for our project.

The feature we are planning to implement from it is, "Route53" supports full, end-to-end DNS resolution over IPv6 and recursive DNS resolvers on IPv6 networks can use either IPv4 or IPv6 transport to send DNS queries to Amazon Route 53. Another feature we wish to implement is the programmatic access. Route 53 gives programmatic access to the service that allows customers to modify DNS records via web service calls.

Customers create "hosted zones" that act as a container for four name servers. The name servers are spread across four different TLDs. Route 53 behaves as a DNS resolver between these hosted zones. Every hosted zone has its own DNS resolver which functions within the hosted zone. Customers are given control to add, delete, and change any DNS records in their hosted zones.

Route 53 also provides latency based routing and weighted routing, the former one does the traffic management such that it will send the requests to the server which will respond with a lower latency, for example, route requests based on the geographical location of the requesting and the destination server. With the later feature, we can route more requests to the server which has more weight and thus do the load balancing for the requests it receives. We try to emulate the latency based routing feature in our project.

2. Kube DNS

Kube DNS is a service provided by Kubernetes that allows the user to schedule a DNS Pod and Service on the cluster and configures the objects (Kubelets) to tell individual containers to use the DNS server IP to resolve DNS names.

A *Pod* is the basic building block of Kubernetes and represents a running process on your cluster. It encapsulates an application container storage resources, a unique network IP, and options that govern how the container should run. A Pod represents a unit of deployment: a single instance of an application in Kubernetes, which might consist of either a single container or a small number of containers that are tightly coupled and that share resources.

Kube DNS provides services such as assigning a "Normal" service to a DNS A record that can be used to resolve the cluster IP of the service. In the case of headless services, it assigns a set of IPs of the pods that are selected by the service. It allows the creation of SRV (service) records that can be resolved to the port number and hostname of any service. Another service

offered is the ability to specify the DNS resolution policy on a per pod basis i.e; Default, ClusterFirst, ClusterFirstWithHostNet, etc.

Basic Architecture

We are planning to implement “Hub and Spoke” topology for the DNS hosting servers. For the simplicity purpose let's call the Hub as Central DNS and Spokes as Local DNS

1. Central DNS - Central DNS service in our architecture would have the DNS entries of all the local DNS on each host. So any DNS lookup between two containers in different host have to look up the central DNS and then they will be directed to the following local DNS server.
2. Local DNS - It has DNS entries for all the servers running on the containers within the same host.
3. On-Premise (Private Network) - the On-premise network is a separate private network which has a function to look up for a service on the public network

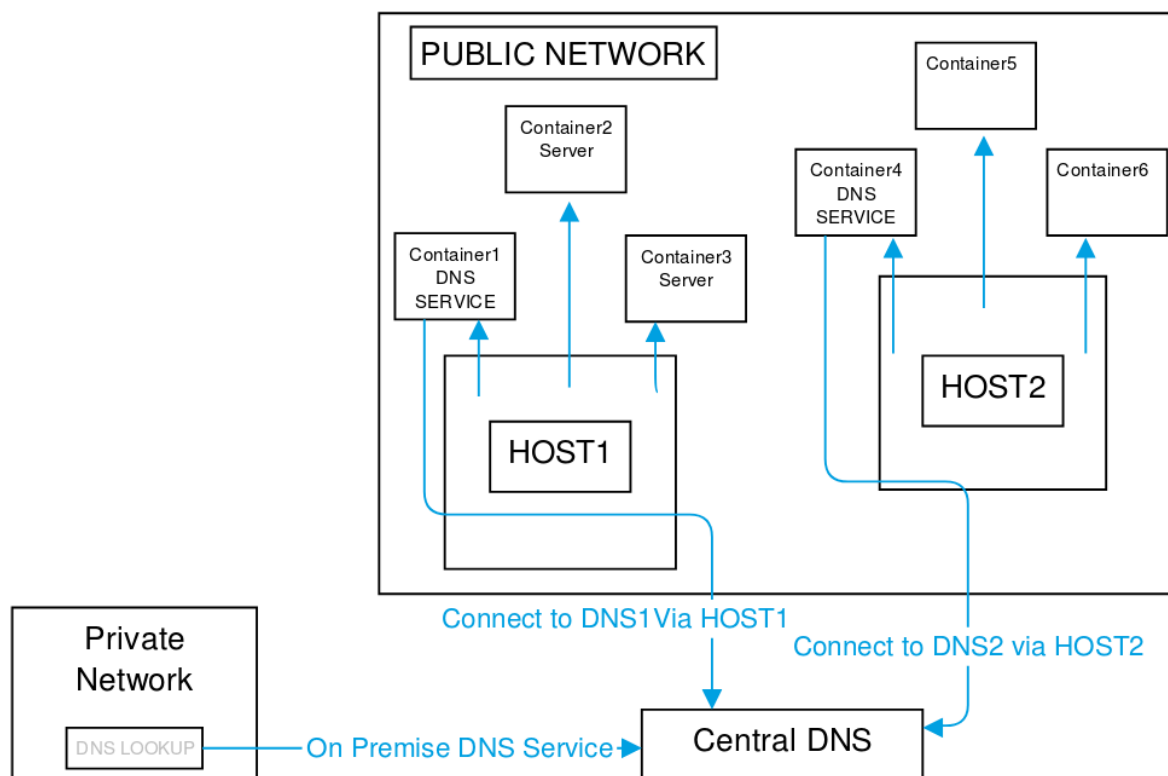


Fig 1. Basic Architecture

Feature list implementation proposal:

Functional features :

1. Configurability: User can configure the DNS by adding the address as either primary or secondary depending on his/her choice. A user can also specify IPV4 or IPV6 address.
2. Managing DNS lookup table: If a server is terminated for a container then remove the DNS entry from the local DNS server or add a new entry if any.
3. Zone (Host) based lookup: Registering a service on a server (container) would have a domain name of the form (name of the service / zone name (where zone name is name of the zone where the server launching the service is)) So in that way central DNS would not have to look up for the given request in every local DNS instead direct it to the specific zone. So in this way we provide smart DNS routing with central DNS having metadata of the available DNS and the domains it is hosting.

Management features :

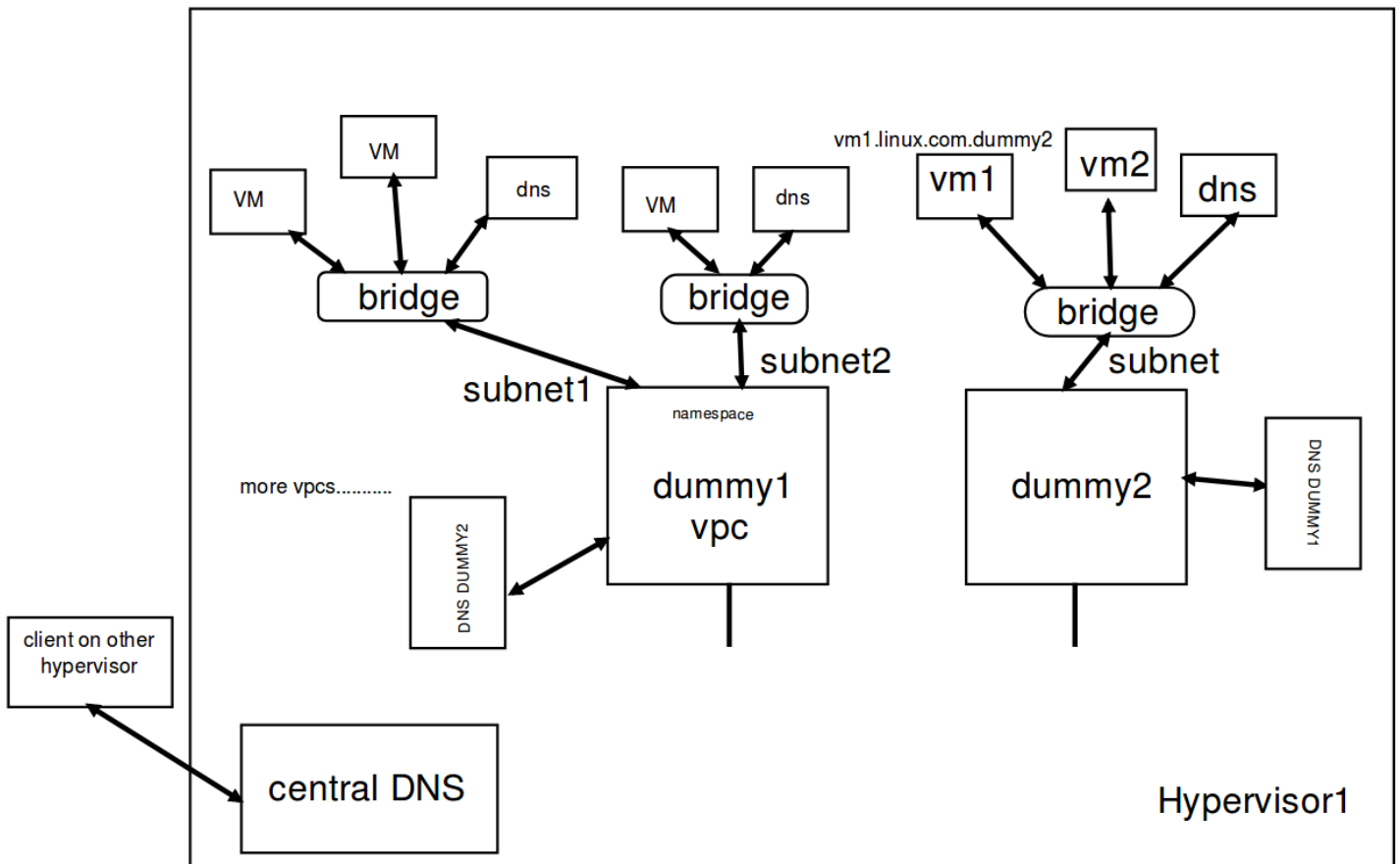
1. Availability: Since there will be at least two DNS servers provided, we can achieve availability very easily.
2. Security: Providing firewall services for security of DNS servers to avoid denial of service attack.
3. Reliability: If a DNS server goes down then utilizing central DNS to get information of the DNS service and restarting it.
4. Performance :
 - A. Caching DNS of containers that are in a different zone, to local DNS servers to avoid looking up in the central server for faster performance. These cached DNS records will have a TTL and expire after that time.
 - B. Latency-based Routing :
 - If an application is hosted in multiple hosts, we can improve performance by serving the requests with a response that provides the lowest latency.
 - To use latency-based routing, we can create latency records. When a DNS receives a query for user's domain it determines which region gives the user the lowest latency, and then saves this as a latency record and responds with the value from the selected record, such as the IP address for a web server.

References:

1. <https://searchcloudcomputing.techtarget.com/definition/hybrid-cloud>
2. <https://aws.amazon.com/what-are-containers/>
3. <https://kubernetes.io/docs/concepts/services-networking/dns-pod-service/>

Milestone 2:

Architecture



Architecture:

We are providing a cloud service where we can support multiple tenants, each tenant can have multiple VPCs and deploy their network inside our cloud. And also a DNS solution where an on premise network of a specific tenant can do a DNS resolution inside the cloud for its servers and reach them inside his VPC.

We also have some restrictions for the tenants who are hosting their VPC inside our cloud. And the tenant provides us with a structured format of data of what all he needs for his VPCs with the constraints in place.

Our VPC Solution

Every customer demanding a VPC would need to provide a Yaml file mentioning the configurations of their network.

The Yaml file is of the format:

```
VPC:
  VPC1:
    VPC_IP: 10.10.0.0/16
    VPC_name: VPC1
    Subnet1:
      Subnet_IP: 10.10.1.0/17
      Subnet_name: linux.com
      VM_IP: 10.10.1.10/17
      VM_name: c1Sub1VM1
      VM_IP: 10.10.1.20/17
      VM_name: c1Sub1VM2
    Subnet2:
      Subnet_IP: 10.10.2.0/17
      Subnet_name: cloud.com
      VM_IP: 10.10.2.10/17
      VM_name: c1Sub2VM1
```

We have added certain constraints to the creation of the network such as:

- 1) The user can ask for multiple VPCs to be created for themselves.
- 2) Each VPC can have a maximum of 2 subnets allocated to it.
- 3) Each subnet can have any number of VMs as specified by the user.

Our Dns Solution

Stages (For our default network we have dummy1 - cmu and dummy 2 - ncsu)

VMS in cmu

Subnet .com : host1.linux.com.cmu, host2.linux.com.cmu

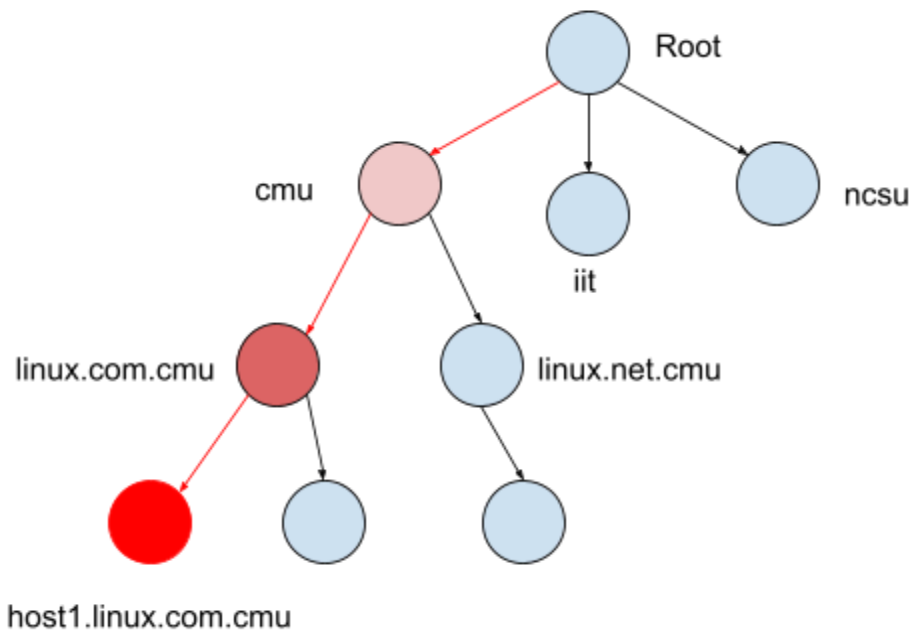
Subnet .net : hosttest.linux.net.cmu

VMS in ncsu

Subnet .com: host3.linux.com.ncsu

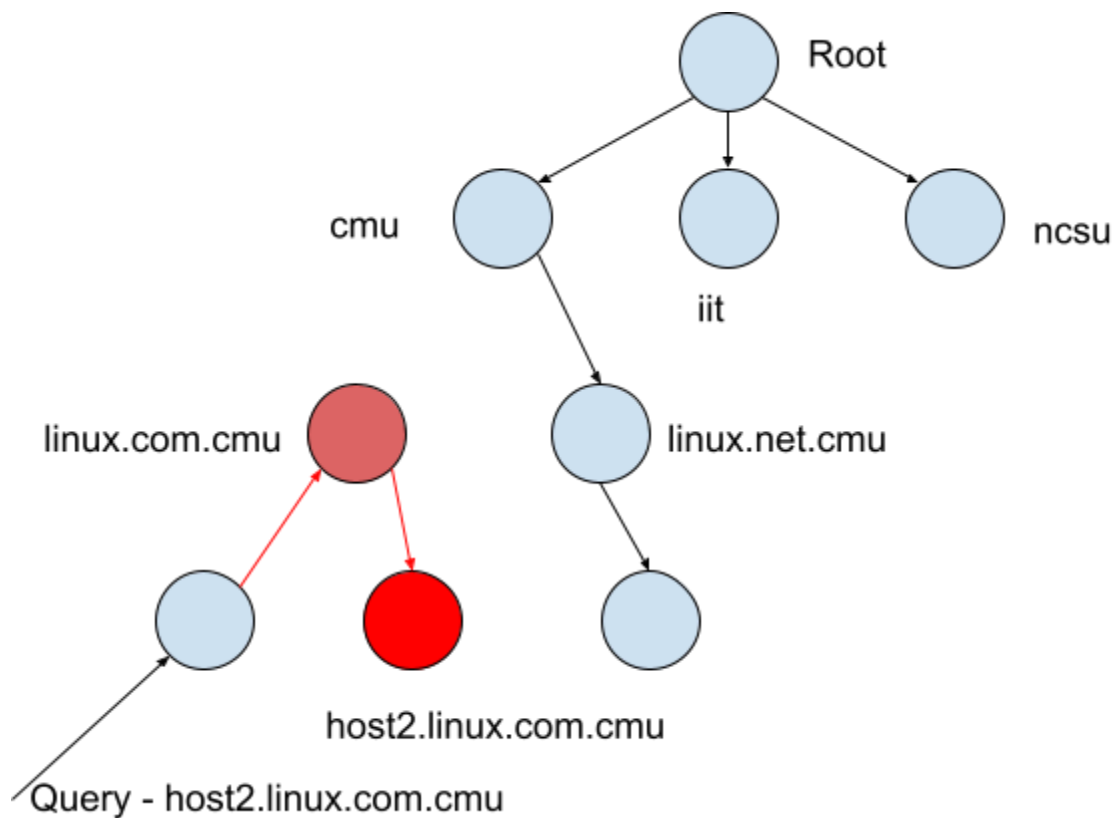
- A DNS request from a customer to a machine within a VPC

Query - host1.linux.com.cmu

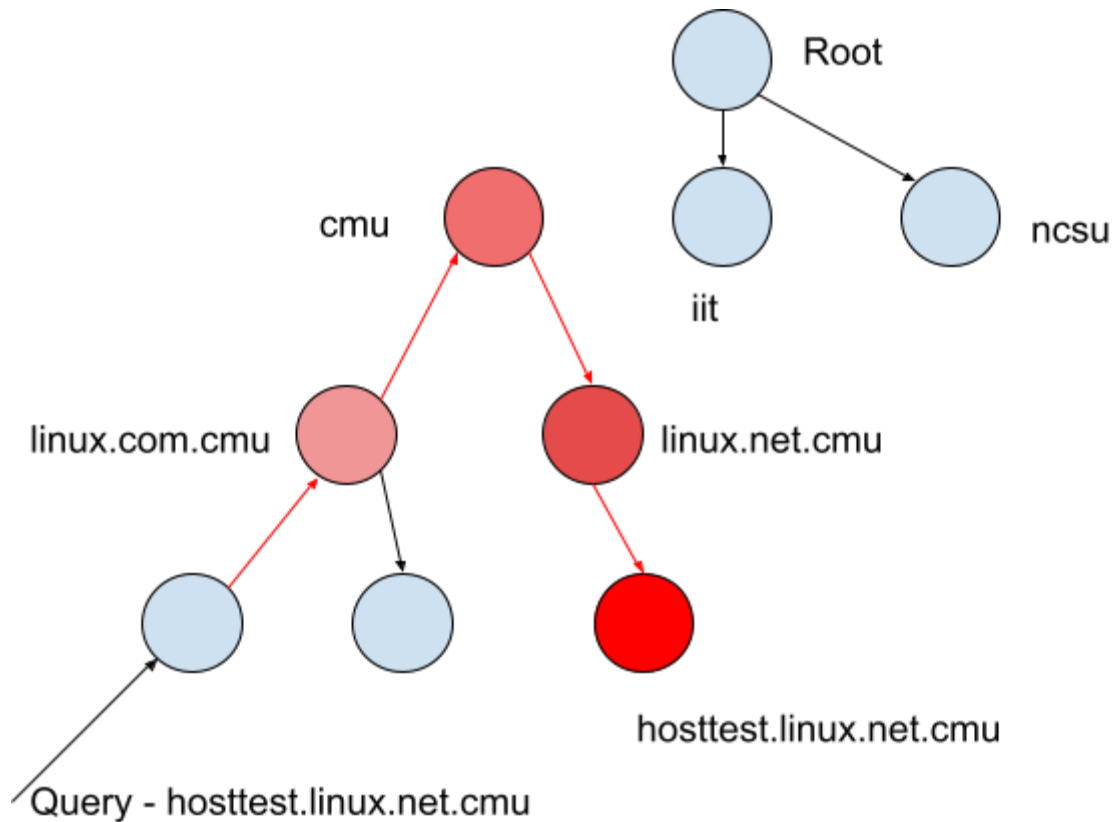


Leaf nodes in all diagrams are the machines itself and not the dns servers and rest are the dns server

- Customer's query is first sent to central dns which has the information of zone cmu, iit, ncsu (suffixes of the domain) (authoritative dns).
 - The central dns sends these queries according to the solution within their zone entry for requested domain
 - Eg: cmu in above case
 - Now zone entry in cmu will forward the request to that particular VPC which belongs to cmu.
 - Now cmu resolves the query by searching the matching zone
 - Eg. linux.com.cmu in above case
 - Now this top level dns forwards the request to linux.com.cmu or linux.net.cmu as per above request.
 - This request gets forwarded to final dns server which gives the entry corresponding to the requested server.
 - FOUND host1.linux.com.cmu
- A DNS request from a machine within a VPC to another machine within the same subnet inside the same VPC (note: VPC here is basically are the institution like CMU, NCSU)



- Now a request for Eg: host2.linear.com.cmu
 - This request is resolved by the subnets dns server which contains the zone information of all the machines with domain "*.linux.com.cmu"
 - So the request matches the zone entry hence looks for host2 within that zone and gets the ip to send the request to.
- DNS request from a machine within a VPC to a machine within the same VPC but different subnet



- Eg. DNS request for hosttest.linear.net.cmu
 - The request first reaches the DNS server of the subnet
 - Now the DNS server does not find any zone matching requested domain hence forwards the request to VPC DNS.
 - Here our dns solution works like a forwarding solution and not a traditional dns solution.
 - I.e. Server just forwards the request to other dns without realizing it was its parent in the above tree.

Automating the above solution

The provider part of the network is setup initially with a Central DNS on the Hypervisor and a dummy DNS and dummy VM image with the necessary packages to enable easy spawning of new respective VMs upon request from the customer.

The yaml file provided by the customers is placed in the location :

Project/roles/createNS/var/ as main.yml

This yaml file is called when the Ansible script is run as input.

An ansible script has been written to do the following tasks:

- 1) Create a Namespace for the customer
- 2) Bridges are created for each subnet
- 3) Create veth pairs to attach the namespace to the subnet bridges.
- 4) VMs are spawned using virsh-install from a pre-created VM image file that has dhclient configured on it.
- 5) Configure DNS conf files (Refer roles/createNS/tasks/chNamed.yml)

The Namespace is created for every VPC. The bridge is created according to every Subnet.

→ We can use the VM created earlier as a DNS server by creating a new Named.conf file from "Soup" named.conf, we can reboot the BIND service to make it as DNS server.

For each new VM created, we follow the following procedure (automated by Ansible):

→ create a copy of the dummy VM image placed at **/var/lib/libvirt/images**

This is done is install VMs without user intervention from a fresh disk image (CentOS).

By default, all VMs are configured with hostname and password as "root".

→ The new image is saved under the name <newvm_name>.img

→virsh-install command is used to create a new VM with the imported image created in the previous step.

→ All new VMs are initially created on the default network named "new" which acts as a dummy network.

→ The script will then ssh into the new VMs and change the IP to reflect the address specified by the customer in the yaml file.

→ The interface is then attached to the respective subnet bridge and removed from the "new" network.

- For every new VPC we set up a dns server within that vpc with a default configuration by launching a VM from existing dns server VM image.
- We open the /etc/named.conf via ssh into the machine and change the required configurations.
- Entries into /var/named/{zone file} is also altered accordingly.
- A DNS server is also launched for every subnet created with the similar changes.
- Lastly for every new VPC we add a forwarding entry in central dns.

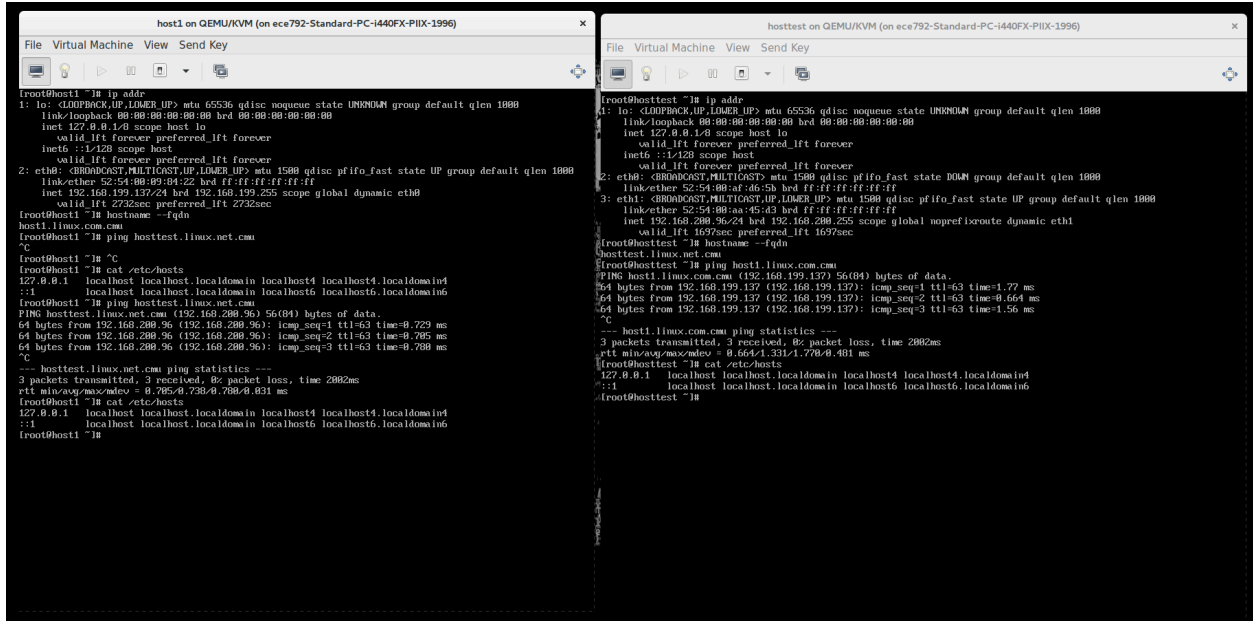


Fig. Request to hosttest.linear.net.cmu which is in different subnet within the same vpc

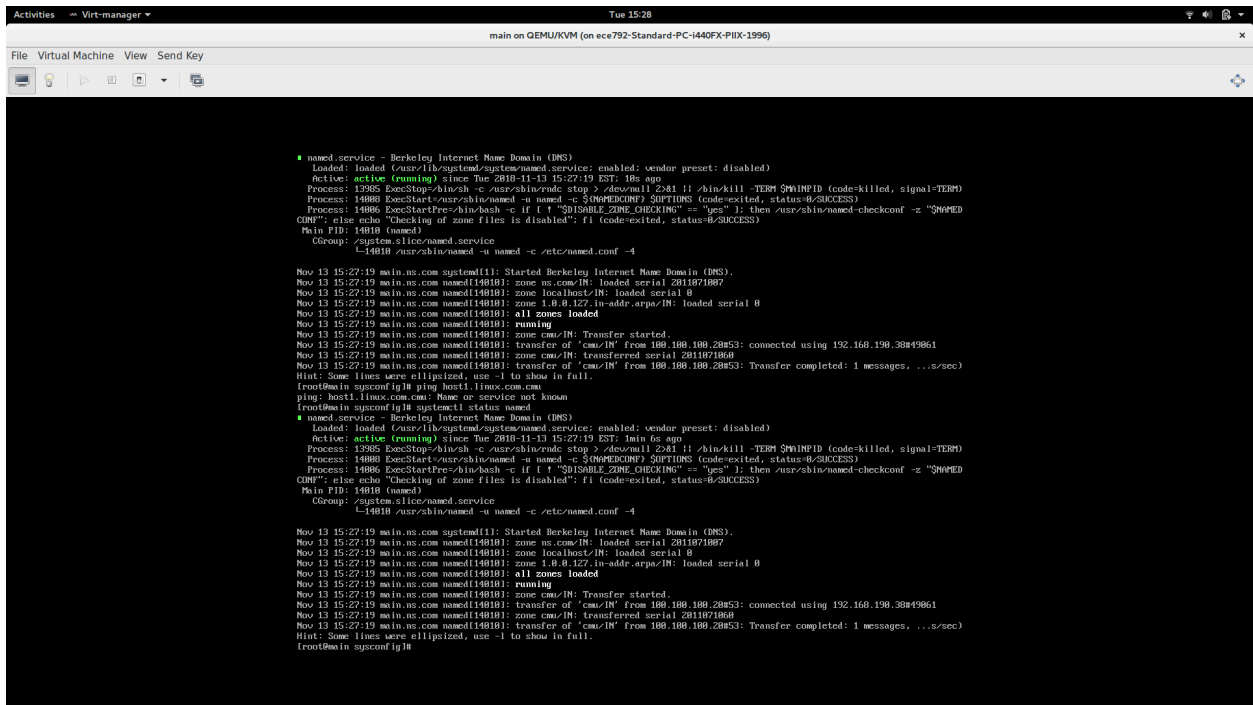


Fig: Status of Central DNS

Final Architecture Of Our Solution

